

Perpsplexity.app — Findings & Impact Report

Scope reviewed: all protocol-owned Sui Move packages on the target network — the main package (pool/curve/launchpad/composite/engine/lending layers), plus the integrated Aftermath and Suilend components — reviewed at bytecode level. **Status:** review complete; two findings block launch; remediation plan below.

This report explains *what* was found, *how each defect actually works*, and *what it means for the project* — funds, availability, and trust. The technical audit report (`report.md`) and evidence files under `audit/` contain the instruction-level detail and reproductions; this is the non-technical-facing version.

1. Executive summary

The platform is a "conviction economy" token-launcher over Aftermath perps and Suilend lending: meme tokens are launched on bonding curves, buyers add quote, pools "graduate" to real constant-product AMMs, and composite pools run perpetual-style engines with funding and winddown machinery. The core financial machinery is engineered carefully — the classic failure classes (integer overflow, share-rounding inflation, capability forgery, reentrancy, dividend over-claim, over-withdrawal, authority confusion) were all specifically verified and not found.

However, **two severe defects affect the core product's first impression:**

1. **F-7 — the graduation ("bond") gate doesn't actually gate anything (Severity 7).** The check meant to prevent a pool from graduating before its funding target is met compares a value that is *always* large enough — so it can never fail, and the pool's meme supply can be burned to zero by anyone, before the raise completes. This enables fund extraction or destruction of any live, unpublished pool in a single transaction.
2. **A21 — launched pools give the creator a withdrawable claim on ~all liquidity (Severity 7).** The "creation liquidity" token granted at launch is never locked or burned. Once the curve is retired (which anyone can force, per F-7), the holder of that token can withdraw essentially the entire pool balance — including all the quote paid in by buyers. Buyers can lose nearly 100% of their deposit with no off-chain recourse, and the payouts are freely transferable (untraceable in practice across fresh wallets).

Two further confirmed issues and a set of minor items round out the picture: a funding-loop availability defect (A20, Severity 6) that can freeze the epoch/reinvest machinery, and a permissionless market-adoption gap (A22, Severity 4) that enables counterfeit markets for any token. Every remaining finding is Severity ≤ 2 , and several suspected classes were resolved as safe.

Bottom line: do not ship the launchpad-facing flow (and do not leave any unbonded pool live) until F-7 and A21 are remediated. Everything else is remediation-level.

2. Findings at a glance

ID	Severity	Type	One-line consequence
F-7 (A17)	7	Wrong gate / dead check	Permissionless premature graduation; ~100% meme-supply burn; fund extraction or grief on any unbonded pool
A21	7	Missing control (unlocked creation LP)	Creator can withdraw ~all pool liquidity incl. buyer quote — launchpad LP-rug
A18 (R-2)	amplifier of F-7	One-tx flash composition	The F-7 extraction completes in a single transaction on composite pools
A20	6	Availability	Funding-rate poisoning ⇒ epoch/reinvest loop frozen + readiness signal spoofed
A22	4	Missing authorization	Anyone can create counterfeit markets typed to any legitimate token
F-1..F-4	1-2	Minor	Dust rounding, UX aborts, grief-by-donation / permissionless grief
T-1..T-4	2-4	Theoretical	Oracle/venue staleness, single-admin trust model — venue-dependent, unproven

3. The two deploy-blocking findings — how they work

3.1 F-7: the graduation gate is dead code

How the design is supposed to work. A pool opens on a bonding curve. Users buy the meme token on the curve. The pool is not meant to "graduate" (turn into a plain AMM) until the curve's *basis* — the quote raised on the curve — reaches the *bond target* set by the launcher. The code even contains a function (`curve::can_bond`) that reads exactly this comparison.

What the code actually does. The guard executed at bond time compares `virtual_quote >= bond_target`. But `virtual_quote` is set, once, at pool creation to be *structurally larger* than `bond_target` (it is the target plus the seed), and nothing ever decreases it. So the comparison is **always true** and its failure branch (abort 3) is unreachable dead code. On top of that, the real intended check (`basis >= bond_target`) is *hard-wired to pass as well*: at creation the basis is required to be ≥ 1000 while the target is capped at 1000 — so even a correct implementation of the intended gate could never fail.

Result: `bond` is effectively permissionless and unconditional on every pool that has a curve (spot: any time, no maturity; composite: once the pool reaches its live lifecycle state). Anyone can fire it at any moment.

How it is exploited (two ways):

1. *Value extraction*: an attacker buys memes on a live, unpublished curve (the curve's virtual depth keeps the price flat, so cost is low), fires `bond` (burns ~100% of the remaining pooled meme supply), then sells the pre-purchased memes back into the now-degenerate AMM. The pool spends its accumulated quote absorbing them at an inflated per-token price — the attacker takes away a large part of the pool's cash. On composite pools the complete attack is **one transaction** (A18).

2. *Grief*: bond a pool you hold nothing in — the meme side is destroyed, the raise is killed, the composite lifecycle is force-ramped regardless of the funding target. Cost: one transaction, no skill.

Impact on the project: every live pool that has not yet graduated is a standing target. Early buyers of any such token are, in effect, buying a position that can be devalued to near zero at any second, on-chain, by anyone. This affects the very first user experience the product creates.

3.2 A21: launch-created liquidity is never locked

How the design is supposed to work. In a fair-launch launchpad, the pool creator's "creation liquidity" position (the permanent minimum-liquidity seed) is normally locked or burned at graduation — otherwise the creator can pull the whole pool out from under buyers the moment the curve retires.

What the code actually does. The creation `LpPosition` is handed to the launcher (via the launch function, adopt, or composite activation) and is never locked, burned, or time-gated. The codebase has a locking library (`locks/lock-tier`) and even uses it for user positions — it is simply **not wired** to launch-created liquidity. The withdrawal function requires only two things: holding that `LpPosition` and a retired curve. Because bond is permissionless (F-7), the retire condition is trivially satisfiable.

Result: the person who launched the pool (or anyone they sold the `LpPosition` to) can, after bond, withdraw ~all of the pool's reserves — the seed **and every quote unit buyers deposited**. The pool is the token's *only* market, so this is essentially the entire real liquidity of the token. The withdrawn coins are ordinary transferable coins — once routed through fresh wallets/contracts there is no practical recovery. Composite pools have the same shape via their activation path. A small unchanged floor (`MIN_LIQ`) remains, too small to matter.

A related gap (A22, Severity 4): `adopt` functions let anyone take control of a pool typed to any legitimate token, without issuer consent or an allowlist — enabling counterfeit/phishing markets that look official.

Impact on the project: if the launchpad ships as-is, a buyer's first encounter with the product can be losing their whole deposit to the creator. That risk: - makes the launchpad incompatible with a "trusted fair-launch" marketing position and with any listing/insurance expectation, - creates an obvious honeypot reputation risk for the brand, - and converts a wedge product (first-time users, low-trust memes) into a liability exactly where it needs trust.

4. Availability finding

4.1 A20: funding-reserve poisoning (Severity 6)

How it works. The composite funding loop refreshes its reserve by reading a quote "market state" (funding rate, timestamp, deltas, engine supply) that the caller is allowed to supply freely, without the pool binding it to the engine market the pool actually trades on. A caller can therefore store a poisoned state — inflated rates/deltas, inconsistent timestamps — that then makes *every honest* refresh and both epoch-funding operations abort (because their internal consistency checks fail). The poisoned values can also spoof the "funding ready" signal, so the reserve looks funded when it is not.

Impact on the project: a single attacker can freeze the funding/reinvest machinery of a composite pool persistently — operations revert, the loop stops accruing, and the readiness signal misleads the UI and operators. No direct theft occurs (dividend claims and principal paths bypass the reserve), so this is a **denial-of-availability** — costly, annoying, and damaging to perceived reliability, but not a loss of funds. Resolution: it heals only if the attacker stops polling.

5. What was verified and found safe

The following classes were actively chased and are **not** defects:

- **Reentrancy / flash-composition:** no in-call reentry is structurally possible (no external-call hooks in value paths; session IDs bind operations; fee and invariant guards hold). One one-tx *composition* amplifier exists as part of F-7 (A18) — the reentry class itself is clean.
- **First-depositor / share-rounding inflation:** resolved safe with extensive simulation (no net value created over hundreds of thousands of mint/redeem cycles).
- **Integer overflow:** u256 math; 300k boundary cases — no wraps.
- **Capability forgery / authority confusion / object mismatch:** systematic checks in place; none found.
- **Fee routing:** exact accounting; creator fee capped $\leq 10\%$, all five destination splits sum exactly.
- **State-5 close-out (R-3/A19):** the engine-path payout is bounded by the pool's own baskets (hot lane, reduce-only share redemption, lending adapter, reserved cap) — no "free cash from the engine's margin" credit exists. Resolved as not a finding.

6. Residual risk & unknowns

- **Live on-chain reproduction of the F-7 sequence** (dev-inspect on an actual chain) was not completed — RPC access was unavailable during the review. The defect is statically verifiable at instruction level, but a live dynamic proof is the remaining confirmation step.
- **Winddown / hibernation sweeps** on composite pools were structurally reviewed (state-gated, basket-bounded) but not exhaustively line-traced; a follow-up pass is recommended before that feature is made prominent.
- **Third-party dependence:** observable valuation, oracle freshness, and engine behavior live behind Aftermath/Suilend — their SLAs are the project's remaining external risk.
- **Single-administrator governance:** pause, fee destinations, and platform fee withdrawal sit under one admin capability (time-locked proposals ≈ 2 days). Not a code defect; a trust-model decision to grow out of (multisig/DAO).

7. Recommended path

Blocking — must fix before any unbonded pool or the launchpad goes live: 1. **F-7:** rescale the bond target to a market-meaningful unit and enforce the *real* gate (`basis >= bond_target`, as `curve::can_bond`) at bond in both spot and composite paths; optionally add a volume/cooldown condition. 2. **A21:** burn or escrow the creation `LpPosition` at launch (or wire the existing lock-tier into pool/composite creation). This makes a creator rug structurally impossible even if a curve is force-retired.

Before launch — high priority: 3. **A20:** bind the refresh call to the pool's own engine market and source rates/timestamps only from that market (as margin ops already do). 4. **A22:** gate adoption on an allowlist/Registry with issuer consent.

Nice-to-have ($\leq$ Severity 2): lot-rounding symmetric closing, UX aborts, and the permissionless grief paths (see `audit/findings/findings.md`).

Process recommendations: given the two find-blocking issues are both in the *trust layer* (permissionless bond + un-locked creation LP) rather than in arithmetic, add explicit "who may do

what, when" invariants to the launchpad spec, and test those gates with adversarial scenarios before launch.

8. Evidence index

- Technical contact-report: `report.md`
- Finding registry w/ instruction offsets & PoCs: `audit/findings/findings.md`
- Attack graph & residual surfaces: `audit/attack_graph.md`
- Launchpad deep-dive (F-7 root cause, A21, A22): `audit/launchpad.md`
- Funding-loop deep-dive (A20): `audit/funding_loop.md`
- Reentrancy/flash + state-5 resolution: `audit/reentrancy_flash.md`
- Disassembly of all reviewed packages: `audit/dis/`
- Simulations/PoCs: `audit/poc/`, `audit/sim.py`, `audit/fuzz.py`

Prepared as an independent review. Neutral methodology; no live-state change was performed during the review.